

01

Overview & approach

The web app already does the hard part — the Quran text pipeline, the audio-sync math, the account/security logic — is real and tested. This plan is about carrying that logic to a native shell, not re-deriving it.

RememberQuran today is a Next.js App Router site with three layers worth distinguishing for a port:

- **Content, stateless:** Quran text, translations, audio, tafsir, morphology, search — all sourced from quran.com/QDC APIs, a couple of CDNs, and one bundled dataset (morphology). No account needed, cacheable, safe to call directly from a mobile client.
- **Account state, Firestore:** users, bookmarks, notes, hifz tracking, goals/streaks, reading position — all in Firestore, all written through `src/lib/firestore/*.ts`, several of which got real correctness fixes this session (transactional count limits, timezone-aware day boundaries).
- **Session/auth, web-shaped:** Auth.js with a Credentials provider and a JWT cookie — a pattern built for a browser, not a mobile client. This is the one layer that shouldn't be ported as-is; see §8.

The recommendation running through this plan: **reuse the content APIs directly, reuse the Firestore data model and write-path logic as-is, and replace the auth transport with native Firebase Auth** rather than porting cookie sessions to a phone.

02

Architecture decisions

Three calls shape everything downstream. Each has a recommendation and the trade-off it's made against.

State management, DI & routing — GetX

The web app's Context layer (`AudioPlayerContext` , `BookmarksContext` , `ReaderSettingsContext` , ...) maps onto a set of `GetXController` s — same shape (a store, a set of actions, derived reactive state), same discipline (e.g. `BookmarksContext` 's per-user cache invalidation becomes a controller re-bound on the auth uid via `Get.find` / `Get.put`). GetX's `.obs` reactive variables + `Obx` widgets are the closest Dart equivalent to a Context's state-and-rerender loop, and because GetX bundles state, dependency injection, and navigation in one package, it also replaces what would otherwise be three separate libraries (state, DI, router) — one mental model end to end, and noticeably less setup boilerplate per feature than a Provider-based stack.

Routing rides on GetX's own navigation (`GetPage` route table + `Get.toNamed`) rather than a separate router package — running two competing navigation paradigms side by side is the one combination worth avoiding; see §04 for the full route table expressed as `GetPage` s.

Backend integration — direct-to-source for content, direct-to-Firestore for accounts

Two backend strategies were on the table:

OPTION	WHAT IT MEANS	VERDICT
Proxy everything through the Next.js API	Flutter calls <code>rememberquran.com/api/*</code> for content and accounts alike, exactly like the web client does.	Adds a network hop and a single point of failure for content that doesn't need one. Right fit only for the handful of routes that do real work (see below).
Direct-to-source (recommended)	Flutter calls <code>quran.com/QDC/CDNs</code> directly for content; uses <code>FlutterFire</code> for accounts.	Matches how a mobile client is supposed to talk to Firebase.

(recommended,

directly for content, uses Flutter fire

(`cloud_firestore` , `firebase_auth`)

directly for accounts.

supposed to talk to Firebase,

cuts latency, and the app keeps

working if `rememberquran.com`

itself has an outage.

Three existing API routes are worth keeping and calling from Flutter rather than reimplementing, because they do real, non-trivial work beyond a pass-through:

- `/api/tafsir/[slug]/[surahId]/[ayahId]` — validates the slug against an allowlist and runs a single-choke-point HTML sanitizer before the content ever reaches a client. Re-implementing an HTML sanitizer in Dart is real, ongoing security-review overhead for zero benefit.
- `/api/asbab/[surahId]/[ayahId]` — same shape: index/allowlist plus a genuine-vs-padding content filter.
- `/api/search` — thin today, but centralizing search is what let this session harden it (dead-abort-controller cleanup) in one place instead of two.

Everything else — chapters, verses, word data, audio file URLs, morphology — is either a public, cacheable GET to `quran.com/QDC` or a static bundled dataset, both safe to call straight from the app.

Offline — bundle the Quran text, stream everything else

The web app has no offline story, and doesn't need one. A mobile Quran app does — it's one of the top reasons people install one. Recommendation: ship the full Uthmani text + word data as a bundled SQLite/Isar database at install time (small — it's text, not audio), so reading, translations, and word meanings work with no connection. Audio and tafsir stay network-backed with an explicit "download this surah" affordance per reciter. Full detail in §7.

03

Functionality inventory

Every feature verified working in the web app this session, grouped the same way the project's own regression checklist groups them, mapped to its data source and the Flutter-side approach.

Core reading

FEATURE	DATA SOURCE	FLUTTER APPROACH
All 114 surahs, Uthmani script + diacritics	QDC <code>verses/by_chapter</code> (<code>qpc_uthmani_hafs</code> field)	Bundled locally after first sync (see §7); render with the real KFGQPC UthmanicHafs font, not a substitute — tajweed/diacritic accuracy depends on it.
Word hover/tap → meaning + transliteration	word-level <code>translation</code> / <code>transliteration</code> fields, bundled with verse data	Tap target opens a bottom sheet (mobile has no hover) with the same content <code>WordMeaningContent.tsx</code> shows.
10 translations, toggle up to 3 at once	QDC + Khattab CDN (id 131, merged client-side — QDC dropped that resource)	Port <code>translations.ts</code> 's registry + <code>MAX_ACTIVE_TRANSLATIONS</code> cap verbatim; same merge-in-app logic for Khattab.
Verse-by-verse vs. continuous reading mode	—	Two list-builder widgets over the same verse data, same split as <code>AyahBlock</code> / <code>ReadingModeView</code> .
Arabic font (Hafs/Amiri) + text size	device-scoped setting	<code>shared_preferences</code> ; bundle both font families as app assets.
Light/dark mode	device-scoped, OS-aware	<code>ThemeMode.system</code> + manual override, mirrored in prefs.
Deep link to a specific ayah	—	Universal Links (iOS) / App Links (Android) → GetX named route → scroll-into-view, retried once the target's data page has loaded (the exact race this session fixed on web).
Copy / share an ayah	—	<code>Clipboard</code> + native share sheet via <code>share_plus</code> — a strictly better fit than the web's clipboard-fallback dance.

Audio & recitation

FEATURE	DATA SOURCE	FLUTTER APPROACH
20 reciters (post-fix; one removed for a	QDC <code>audio/reciters/{id}/audio_files</code>	Port <code>audioSources.ts</code> 's registry as-is, including the removed-id comment.

dead upstream file)		
Persistent mini player across navigation	—	<code>audio_service</code> gives this for free — a foreground/background audio session is the native equivalent of the web's root-mounted <code><audio></code> .
Word-by-word highlight sync	QDC <code>segments</code> field per verse timing	Port <code>wordSync.ts</code> 's binary-search + sanitize logic directly — it's pure Dart-portable math, no DOM dependency.
Click-to-hear word pronunciation	QDC word <code>audio_url</code> , pause-marker-adjusted	Port <code>getWordAudioUrl</code> 's regex-normalization verbatim (verified this session against live data).
Repeat ayah/range, with pause between reps	—	Port the state machine from <code>AudioPlayerContext.tsx</code> (mode/count/pauseMs) onto <code>just_audio</code> 's player; same edge cases already found (manual-nav-cancels-repeat, gap-timer-must-be-cancelled-on-manual-toggle) apply identically.
Playback speed	—	<code>just_audio</code> 's native <code>setSpeed</code> — no Safari-style re-apply-on-load workaround needed, that was an HTML5-audio-specific quirk.
Quran Radio — continuous playthrough, wraps 114→1	—	Port <code>advanceRadio</code> 's wraparound + fail-streak-skip logic; this is also where a lock-screen "next surah" media control becomes trivial via <code>audio_service</code> .

Study tools

FEATURE	DATA SOURCE	FLUTTER APPROACH
Tafsir panel, 5 books	via <code>/api/tafsir/*</code> (kept — see §2)	Bottom sheet / side panel with a book selector, same 5-entry registry.
Taiweed colour	QDC	Port <code>taiweed.ts</code> 's parser + the alian-with-

coding	<code>text_uthmani_tajweed</code> field	fallback logic onto <code>RichText / TextSpan</code> — the alignment strategy (parse rules, map onto the real glyph string) is exactly as necessary in Flutter as it is in the DOM.
Keyword search, Arabic + English	via <code>/api/search</code> (kept — see §2)	Same request shape; render <code></code> -highlighted snippets by porting <code>splitEmHighlights</code> 's regex parser (never raw HTML injection).
Word morphology — root, lemma, grammatical form	bundled 114-file JSON dataset	Ship the same dataset as an asset; port <code>morphologyLabels.ts</code> 's POS/feature dictionaries verbatim — every code in it was verified against the live dataset this session.
Asbab al-Nuzul (reasons for revelation)	via <code>/api/asbab/*</code> (kept — see §2)	Same index-then-fetch pattern; ship <code>asbabIndex.ts</code> 's presence table as an asset so the "available" affordance doesn't need a network round-trip to know whether to show itself.

Accounts & personal features

FEATURE	DATA SOURCE	FLUTTER APPROACH
Register / login / logout	Firebase Auth (replacing NextAuth — see §8)	<code>firebase_auth</code> 's native email/password flow.
Password reset	Firebase Auth built-in reset email	Native SDK call — the custom token/Resend flow this session hardened becomes unnecessary, Firebase already does it securely.
Change email / password	Firebase Auth + Firestore	Native SDK re-auth-then-update flow; still bump the shared <code>passwordChangedAt</code> convention if any cookie-based web session needs to keep being revoked.
Bookmarks + collections	Firestore <code>users/{uid}/bookmarks</code> , <code>bookmarkCollections</code>	Port <code>bookmarks.ts / bookmarkCollections.ts</code> to the Flutter Firestore SDK near-verbatim — transactions are a first-class primitive there too, including the TOCTOU fixes

Private per-ayah notes	Firestore <code>users/{uid}/notes</code>	Same port; the count-limit transaction fix carries over directly.
"Continue where you left off"	Firestore <code>lastPosition</code> on the user doc	Same write-throttle pattern (dwell + interval), triggered from scroll position in the reading list widget.
Reading progress, daily goals, streaks	Firestore <code>progressEvents</code> , <code>activeGoal</code> , <code>streak</code>	Port <code>goals.ts</code> / <code>progress.ts</code> as-is — including this session's timezone-aware day-boundary fix and the range-merge (no inflation) fix. Device timezone is trivially available natively, no cookie-sync trick needed.
Media Maker — ayah image card export	—	<code>RepaintBoundary.toImage()</code> on the same preview widget used on-screen, then <code>share_plus</code> — actually simpler than the web's CORS-sensitive canvas rasterization.
Guest mode — reading/audio/study work with no account	—	Same rule as web: nothing in the content path should touch <code>firebase_auth</code> 's current-user state; only the account-scoped screens gate on it.

Expansion

FEATURE	DATA SOURCE	FLUTTER APPROACH
"Hide Arabic" memorisation mode	device- + session-scoped state	Blur/opacity overlay + tap-to-reveal per ayah, same per-verse <code>Set</code> model as <code>ReaderSettingsContext</code> ; reset on surah change.
Hifz progress tracker, by surah/juz	Firestore <code>memorisedAyahs</code> + bundled juz-boundary table	Port <code>hifz.ts</code> (with this session's transactional count fix) and <code>juz.ts</code> 's boundary data — juz divisions cross surah boundaries, so the boundary table has to be exact.

NOT CARRIED OVER

Two web-only routes don't need a mobile equivalent: `/api/aa/ayah` (social-preview

https://claude.ai/code/artifact/c7045960-6542-4a5b-b838-f9524bd8da27?via=auto_preview

meta images — a web sharing concern) and `/api/health` (ops monitoring for the web deployment).

04

Screens & navigation

A GetX `GetPage` route table that mirrors the App Router's actual URL structure, so a web deep link and a mobile deep link resolve to the same place.

ROUTE	SCREEN	NOTES
<code>/</code>	Home — surah directory, ayah-of-the-day, continue-reading card	Ayah-of-the-day ports directly: it's a static, deterministic 20-entry table keyed by UTC day index, zero network dependency.
<code>/:surahId</code>	Reader	Bottom app bar hosts display-mode / font / translation controls instead of a settings panel flyout.
<code>/:surahId/:ayahId</code>	Reader, deep-linked and scrolled to that ayah	
<code>/search</code>	Search	
<code>/radio</code>	Quran Radio	Natural fit for a dedicated "Now Playing"-style full-screen player on mobile.
<code>/media-maker</code>	Ayah card designer	Launched from a share/export action rather than being a standalone tab.
<code>/login</code> , <code>/register</code> , <code>/reset</code>	Auth flows	Native Firebase Auth UI patterns, not the web's form components.

<code>/account</code> , <code>/account/bookmarks</code> , <code>/account/notes</code> , <code>/account/hifz</code> , <code>/account/goals</code> , <code>/account/progress</code> , <code>/account/settings</code>	Account tab	Reasonable to collapse into one tab with sub-navigation rather than mirroring six separate routes.
---	-------------	--

05

Package plan

Concrete choices, one per concern — no bake-off, these are the well-worn defaults for each job.

CONCERN	PACKAGE	WHY
State + DI + routing	<code>get</code> (GetX)	One package for reactive state (<code>.obs</code> / <code>Obx</code>), dependency injection (<code>Get.put</code> / <code>Get.find</code>), and named-route navigation with deep-link support — the closest structural match to the existing Context layer, with less setup per feature than a three-package stack.
Local key-value cache (optional)	<code>get_storage</code>	GetX's own lightweight storage — a natural pairing if the team wants to stay fully in the GetX ecosystem; <code>shared_preferences</code> (below) is the more conservative default and works identically either way.
Audio	<code>just_audio</code> + <code>audio_service</code>	Background playback, lock-screen controls, media-session integration — the mobile-native version of everything <code>AudioPlayerContext.tsx</code> hand-builds for the web <code><audio></code> element.
Networking	<code>dio</code>	Interceptors make the QDC/quran.com/CDN split (three base URLs, same as <code>quranApi.ts</code>) easy to keep organized.
Accounts + data	<code>firebase_auth</code> , <code>cloud_firestore</code> , <code>firebase_app_check</code>	Native session model; App Check replaces the web's IP-based rate limiting with device attestation, a stronger primitive mobile gets for free.
Local prefs	<code>shared_preferences</code>	Direct equivalent of the web's <code>useLocalStorage</code>

(device-scoped)		— reader settings, hide-Arabic state, sidebar prefs.
Offline Quran store	<code>drift</code> (SQLite) or <code>isar</code>	Bundled verse/word/morphology data; see §7.
Image export + share	<code>share_plus</code>	Native share sheet — strictly simpler than the web's canvas-taint/CORS concerns.
Fonts	bundled assets: Amiri, Amiri Quran, KFGQPC Uthmanic Hafs	Same three families the web ships; tajweed/diacritic rendering accuracy depends on using the real fonts, not a system Arabic fallback.

06

Project file structure

A feature-first `lib/` tree — one folder per screen area,GetX's `controller` holding the reactive state a Context provided on web, `bindings` doing the `Get.put` wiring a Provider tree did implicitly. Comments mark where each piece's logic comes from in the current `src/`.

```
lib/
├─ main.dart                                # MaterialApp root + initial bindings
├─ app/
│   ├─ routes/
│   │   ├─ app_pages.dart                  # GetPage table – mirrors src/app/ URL st
│   │   └─ app_routes.dart                # route name constants
│   └─ bindings/
│       └─ initial_binding.dart            # controllers Get.put() at startup
├─ core/
│   ├─ theme/
│   │   ├─ app_theme.dart                  # light/dark ThemeData ← src/app/globals.
│   │   ├─ app_colors.dart
│   │   └─ app_fonts.dart                  ← src/lib/fonts.ts, readerFonts.ts
│   ├─ network/
│   │   ├─ dio_client.dart                 # base Dio + interceptors
│   │   └─ api_endpoints.dart              ← src/lib/quranApi.ts base URLs (v4 / QDC
│   ├─ constants/
│   │   └─ app_constants.dart              # TOTAL_SURAHS, MAX_ACTIVE_TRANSLATIONS,
│   └─ utils/
│       ├─ verse_key.dart                  ← src/lib/quran/verse-key.ts
│       └─ local_dav.dart                  ← src/lib/progress/date.ts (tz-aware dav
```

```

└─ juz.dart ← src/lib/quran/juz.ts

└─ data/
  └─ models/ ← src/types/quran.ts, audio.ts, study.ts
    └─ chapter.dart / verse.dart / word.dart / translation.dart
    └─ reciter.dart / tafsir_resource.dart / morphology_entry.dart
    └─ bookmark.dart / note.dart / hifz_entry.dart / goal.dart / streak.dart
  └─ datasources/
    └─ remote/
      └─ quran_remote_ds.dart ← src/lib/quranApi.ts (v4 + QDC + Khattab
      └─ audio_remote_ds.dart ← src/lib/audioApi.ts, audioSources.ts
      └─ tafsir_remote_ds.dart # calls kept /api/tafsir proxy
      └─ asbab_remote_ds.dart # calls kept /api/asbab proxy
      └─ search_remote_ds.dart # calls kept /api/search proxy
      └─ morphology_local_ds.dart ← src/lib/morphologyApi.ts (reads bundled
    └─ local/
      └─ quran_db.dart # drift/isar schema – bundled offline ver
      └─ prefs_ds.dart ← src/hooks/useLocalStorage.ts
      └─ firestore_ds.dart # thin cloud_firestore wrapper
  └─ repositories/
    └─ quran_repository.dart
    └─ audio_repository.dart
    └─ study_repository.dart # tafsir + morphology + asbab + search
    └─ bookmarks_repository.dart ← src/lib/firestore/bookmarks.ts, bookmar
    └─ notes_repository.dart ← src/lib/firestore/notes.ts
    └─ hifz_repository.dart ← src/lib/firestore/hifz.ts
    └─ goals_repository.dart ← src/lib/firestore/goals.ts, progress.ts
    └─ auth_repository.dart ← src/auth.ts (firebase_auth-backed)

└─ features/
  └─ home/
    └─ controllers/home_controller.dart
    └─ views/home_view.dart ← src/components/surah-list/*
    └─ widgets/
      └─ ayah_of_the_day_card.dart ← src/lib/quran/ayah-of-the-day.ts
      └─ continue_reading_card.dart ← ContinuePrompt.tsx
      └─ surah_list_tile.dart
  └─ reader/
    └─ controllers/
      └─ reader_controller.dart ← QuranReader.tsx, SurahContentContext
      └─ reader_settings_controller.dart ← ReaderSettingsContext.tsx
    └─ views/
      └─ surah_reader_view.dart
      └─ reading_mode_view.dart ← ReadingModeView.tsx (continuous mode)
    └─ widgets/
      └─ ayah_block.dart ← AyahBlock.tsx
      └─ arabic_word.dart ← ArabicWord.tsx (tap sheet, no hover on
      └─ word_meaning_sheet.dart ← WordMeaningContent.tsx
      └─ translation_block.dart
      └─ hideable_arabic.dart ← HideableArabic.tsx (memorisation mode)
      └─ tajweed_text.dart ← src/lib/tajweed.ts, ported to TextSpan

```

```

|   |─ audio/
|   |   |─ controllers/audio_controller.dart ← AudioPlayerContext.tsx (repeat/radi
|   |   |─ services/audio_handler.dart      # audio_service background handler
|   |   |─ views/
|   |   |   |─ mini_player.dart             ← AudioPlayerBar.tsx
|   |   |   |─ radio_view.dart             ← RadioPanel.tsx
|   |   |   └─ widgets/reciter_selector.dart / repeat_controls.dart / speed_control.dar
|   |─ study/
|   |   |─ controllers/study_panel_controller.dart ← StudyPanelContext.tsx
|   |   |─ views/tafsir_view.dart / word_detail_view.dart / asbab_view.dart
|   |   |   └─ widgets/tafsir_book_selector.dart
|   |─ search/
|   |   |─ controllers/search_controller.dart
|   |   |   └─ views/search_view.dart
|   |─ media_maker/
|   |   |─ controllers/ayah_card_controller.dart
|   |   |   └─ views/ayah_card_designer_view.dart # RepaintBoundary export, not canvas
|   └─ account/
|       |─ controllers/
|       |   |─ auth_controller.dart         ← src/auth.ts, AuthNav.tsx
|       |   |─ bookmarks_controller.dart    ← BookmarksContext.tsx
|       |   |─ notes_controller.dart / hifz_controller.dart / goals_controller.dart
|       |   └─ views/
|       |       |─ login_view.dart / register_view.dart / reset_view.dart
|       |       |─ account_home_view.dart / settings_view.dart
|       |       └─ bookmarks_view.dart / notes_view.dart / hifz_view.dart / goals_view.
|
|─ shared/widgets/
|   |─ app_scaffold.dart                    # bottom-nav shell ← NavBar.tsx / Footer.
|   |─ soft_gate_sheet.dart                ← SoftGateDialog.tsx (guest gating prompt
|   └─ loading_skeleton.dart

assets/
|─ fonts/
|   |─ Amiri-Regular.ttf
|   |─ AmiriQuran-Regular.ttf
|   └─ UthmanicHafs.ttf
└─ data/
    |─ morphology/                        ← public/data/morphology/v1/{1..114}.json
    |─ juz_boundaries.json                ← src/lib/quran/juz.ts
    |─ asbab_index.json                   ← src/data/asbab-index.json
    └─ ayah_of_the_day.json               ← src/lib/quran/ayah-of-the-day.ts

test/
|─ unit/
|   |─ word_sync_test.dart                # pause-marker regex – verified live agai
|   |─ tajweed_parser_test.dart
|   |─ local_day_test.dart                # tz day-boundary math – verified live th
|   |   └─ range_merge_test.dart          # non-contiguous read merge – verified li
└─ widget/
    └─ ayah_block_test.dart

```

PORTING NOTE

The `data/repositories` layer is the highest-leverage part of this tree — it's a near-literal translation of `src/lib/firestore/*.ts`, including the transactional fixes from this session (count-limit races in bookmarks/notes/hifz, the collection-existence race in bookmarks). Firestore transactions are the same primitive in the Flutter SDK, so those fixes aren't re-derived, just re-typed.

07

Data & offline strategy

The one place this plan asks for more than a straight port — mobile users expect offline reading in a way web users don't.

- **Bundle at install:** full Uthmani text, word-level translation/transliteration, and the morphology dataset, as a local SQLite/Isar store — this is text, comfortably a few MB, not audio. Reading, word meanings, and morphology all work with the device in airplane mode from first launch.
- **Stream, with an explicit download:** audio (per reciter, per surah — offer a "download for offline" action, mirroring what most recitation apps already train users to expect) and tafsir (smaller, cache-as-you-go is enough).
- **Sync, not replace:** translations can still be fetched live for anything beyond the bundled default pair, same as the web's progressive-load pattern — no need to bundle all 10 translations for every user by default.
- **Account data** (bookmarks, notes, hifz, progress) uses Firestore's own offline persistence (`cloud_firestore` ships this out of the box), which already gives write-then-sync behavior for free — genuinely less work than the web app's optimistic-update-with-manual-rollback pattern in `BookmarksContext.tsx`.

08

Auth & security porting

Every security property this session found or hardened, and where it lands in the native-Firebase model.

WEB PROPERTY (THIS SESSION)	MOBILE EQUIVALENT
Cryptographically random, single-use, server-validated password-reset tokens	Handled entirely by Firebase Auth's built-in reset flow — nothing to port, nothing to re-audit.
Timing-safe reset-request response (fixed this session)	Not applicable — no custom endpoint to leak timing on.
Session revocation on password/email change (<code>passwordChangedAt</code> , added this session)	Firebase Auth invalidates refresh tokens on password change natively; still worth keeping <code>passwordChangedAt</code> as a Firestore field if the web app's cookie sessions need to keep being revoked from a shared backend.
Per-IP / per-email login rate limiting (Firestore-transaction-based)	Firebase App Check + Firebase Auth's own abuse protection cover most of this; the identical Firestore-transaction limiter (<code>rateLimit.ts</code>) still ports directly for any custom callable functions that need it.
TOCTOU-safe writes for bookmarks / notes / hifz (fixed this session)	Firestore transactions are the same primitive in the Flutter SDK — every fix ports as a direct transaction-for-transaction translation.
Bookmark/note privacy via Firestore path scoping (<code>users/{uid}/...</code>)	Identical — enforce with Firestore Security Rules instead of trusting a server-side session check, which is the mobile-idiomatic version of the same guarantee.

DECIDED — SEE §12

Web and mobile share one Firebase project — one account works everywhere. That means the web app's NextAuth Credentials provider eventually wants to verify Firebase Auth tokens instead of its own bcrypt/JWT pair — a bigger change to the web app than anything mobile-only, worth scoping as its own follow-up rather than a Flutter-launch blocker.

Phased roadmap

Same phase structure as the project's own M1–M6 milestones, because the underlying feature set is already organized that way.

Phase 0 **Foundation**

Project scaffold, GetX wiring (controllers, bindings, named routes), Firebase project setup (or shared project, per §8), design tokens/theme, font bundling.

Exit: empty shell app builds and runs on a physical iOS and Android device.

Phase 1 **Core reading (M1)**

Surah directory, reader (both display modes), translations, word tooltips, font/size/theme settings, deep linking, copy/share.

Exit: a guest can read the whole Quran offline, in any bundled translation, and share an ayah.

Phase 2 **Audio & recitation (M2)**

Player service, reciter selection, word-highlight sync, click-to-hear, repeat/pause, speed, Radio.

Exit: background playback with lock-screen controls survives an app backgrounding and a phone call interruption.

Phase 3 **Study tools (M3)**

Tafsir, tajweed, search, morphology, asbab.

Exit: all 5 tafsir books, tajweed colouring, and morphology labels verified against the same live-data spot-checks this session ran.

Phase 4 **Accounts (M4)**

Firebase Auth flows, bookmarks/collections, notes, continue-reading, goals/streaks, Media Maker.

Exit: full account lifecycle — register, use every account feature, change password, delete account — with no orphaned Firestore data at any step.

Phase 5 **Expansion (M5)**

Hide-Arabic memorisation mode, hifz tracker by surah/juz.

Exit: full functional parity with the web app's regression checklist.

Phase 6 **Store readiness (M6)**

Performance pass, offline-download UX, App Store / Play Store listing + review-guideline compliance, crash reporting.

Exit: submitted to both stores.

10

Step-by-step implementation guide

§09 said what each phase delivers and how to know it's done. This is the ordered "do this, then this" underneath it — same six phases, broken into concrete actions against the file tree in §06.

Phase 0 Foundation

- 1 `flutter create rememberquran --org com.rememberquran --platforms ios,android`
- 2 Add core packages to `pubspec.yaml`: `get`, `dio`, `cloud_firestore`, `firebase_auth`, `firebase_core`, `firebase_app_check`, `shared_preferences`, `just_audio`, `audio_service`, `share_plus`, `drift` + `drift_dev` / `build_runner`, `sqlite3_flutter_libs`.
- 3 Run `flutterfire configure` against the chosen Firebase project (same project as web, or a new one — the call flagged in §12) to generate `firebase_options.dart`.
- 4 Scaffold the empty `lib/` tree from §06 — folders and placeholder files, nothing implemented yet.
- 5 Wire `main.dart`: `WidgetsFlutterBinding.ensureInitialized()` → `Firebase.initializeApp()` → `GetMaterialApp(initialRoute:, getPages: AppPages.routes)`.
- 6 Build `core/theme/app_theme.dart` — port the palette from `globals.css`'s custom properties into light/dark `ThemeData`.
- 7 Bundle Amiri, Amiri Quran, and KFGQPC Uthmanic Hafs under `assets/fonts/`, register them in `pubspec.yaml`.
- 8 Confirm the empty shell builds and runs on a physical iOS device and an Android device.

Phase 1 Core reading (M1)

- 1 Port `quranApi.ts` to `quran_remote_ds.dart` — the same three base URLs (v4, QDC, Khattab CDN) and the same `WORD_FIELDS` / `VERSE_FIELDS` query params.
- 2 Define the `drift` schema in `quran_db.dart` mirroring `types/quran.ts`'s `Chapter` / `Verse` / `Word`; write the one-time seed-from-bundle-or-first-sync routine.
- 3 Build `quran_repository.dart` as the single read path every screen goes through —

local DB first, remote refresh behind it.

- 4 Implement `home_view.dart` + `home_controller.dart` : surah directory, ayah-of-the-day card (port the static 20-entry table and UTC day-index function verbatim).
- 5 Implement `reader_controller.dart` + `surah_reader_view.dart` for verse-by-verse mode first, via `ayah_block.dart` .
- 6 Add `reading_mode_view.dart` for continuous mode, sharing `arabic_word.dart` with verse-by-verse mode — mirrors the web's shared-component split between `ArabicLine.tsx` and `ArabicWord.tsx` .
- 7 Implement `arabic_word.dart` 's tap-to-reveal bottom sheet (`word_meaning_sheet.dart`) — tap is the only affordance on mobile, no hover state to build.
- 8 Implement `reader_settings_controller.dart` — font, size, display mode, theme — backed by `shared_preferences` .
- 9 Wire deep linking: configure Universal Links / App Links against `rememberquran.com` , add the `GetPage` route for `/:surahId/:ayahId` , and port the retry-once-data-arrives fix from `QuranReader.tsx` (§03) so a link to a verse past page 1 of a long surah still resolves.
- 10 Add copy/share via `share_plus` .

Checkpoint: read the whole Quran offline, in both display modes, in the default translation pair.

Phase 2 Audio & recitation (M2)

- 1 Port `audioSources.ts` 's 20-reciter registry to `reciter.dart` .
- 2 Build `audio_handler.dart` on `audio_service` + `just_audio` — background session, lock-screen controls, media-session metadata.
- 3 Port `wordSync.ts` 's sanitize + binary-search logic into `core/utils` — pure math, a syntax port not a redesign.
- 4 Port `getWordAudioUrl` 's pause-marker regex normalization verbatim — verified against live QDC data this session, don't re-derive it.
- 5 Build `audio_controller.dart` 's repeat state machine (mode/count/pauseMs), including this session's two fixes: manual navigation cancels an active repeat, and manual play/pause or scrubbing during a pause gap cancels the pending auto-resume timer.
- 6 Build `radio_view.dart` plus the wraparound (114→1) and fail-streak-skip logic from `advanceRadio` .
- 7 Wire `mini_player.dart` as a persistent bar above the tab shell (`app_scaffold.dart`) — mounted once, not scoped to any single screen.

Checkpoint: background playback with lock-screen controls survives an app

backgrounding and a phone-call interruption.

Phase 3 Study tools (M3)

- 1 Wire `tafsir_remote_ds.dart` to the kept `/api/tafsir/*` proxy (§02) — no client-side HTML sanitizer to build.
- 2 Same pattern for `asbab_remote_ds.dart` → the kept `/api/asbab/*` proxy; bundle `asbab_index.json` as an asset so the "available" indicator needs no network round-trip.
- 3 Bundle the 114-file morphology dataset under `assets/data/morphology/`; port `morphologyLabels.ts` 's POS/feature dictionaries verbatim.
- 4 Port `tajweed.ts` 's parser + align-with-fallback logic into `tajweed_text.dart`, targeting `TextSpan` instead of DOM spans.
- 5 Wire `search_remote_ds.dart` to the kept `/api/search` proxy; port `splitEmHighlights` 's regex parser for highlighted snippets — styled `TextSpan` s, never raw HTML injection.

Checkpoint: all 5 tafsir books, tajweed colouring, and morphology labels spot-checked against the same live-data verification this session ran.

Phase 4 Accounts (M4)

- 1 Enable email/password sign-in in the Firebase console; build `auth_controller.dart` + `login_view.dart` / `register_view.dart` / `reset_view.dart` on the native SDK flows.
- 2 Port `bookmarks.ts` / `bookmarkCollections.ts` to `bookmarks_repository.dart` with `cloud_firestore` transactions — carry over this session's fixes: the collection-existence check moved inside the write transaction, and the count-limit check made transactional.
- 3 Port `notes.ts` to `notes_repository.dart`, same transactional count-limit pattern.
- 4 Port `goals.ts` / `progress.ts` to `goals_repository.dart` — the timezone-aware day boundary is simpler here than on web (the device's own local time is directly available, no cookie-sync trick needed), and carry over the range-merge fix (store disjoint ayah ranges, never a min/max span).
- 5 Implement `continue_reading_card.dart` off `lastPosition`, same dwell-then-throttle write pattern as `ProgressTracker.tsx`.
- 6 Implement Media Maker's `ayah_card_designer_view.dart` via `RepaintBoundary.toImage()` + `share_plus`.
- 7 Gate every account screen behind `auth_controller.dart` 's auth state; leave every reading/audio/study path untouched by it — guest mode is the default, not a special case to remember.

Checkpoint: full account lifecycle — register, use every account feature, change password, delete account — with no orphaned Firestore data at any step.

Phase 5 Expansion (M5)

- 1 Implement `hideable_arabic.dart` — blur/opacity + tap-to-reveal, a per-ayah `Set<String>` in `reader_settings_controller.dart`, cleared on surah change.
- 2 Port `hifz.ts` (with its transactional count fix) and `juz.ts` 's boundary table into `hifz_repository.dart`; build the surah- and juz-tab progress views.

Checkpoint: full parity against `docs/regression-checklist.md`.

Phase 6 Store readiness (M6)

- 1 Performance pass: cold-start time, scroll jank on a mid-tier Android device, audio-start latency.
- 2 Build the offline-download UX (per-surah, per-reciter "download for offline") from §07.
- 3 Wire Firebase Crashlytics before the first TestFlight / internal-testing build.
- 4 App Store Connect + Play Console listings — screenshots, privacy nutrition labels, Play Data Safety form.
- 5 Submit to both stores.

11

Testing & QA

Extend `docs/regression-checklist.md` rather than starting a new one — the feature list doesn't change, only the platform-specific rows added below it.

- Offline mode: airplane-mode read-through of a bundled surah, word tooltips, morphology.
- Background audio: phone call interruption, Bluetooth device switch mid-playback, lock-screen control responsiveness.
- Deep links: a shared ayah link opens the app (not the browser) and scrolls to the right verse, cold-start and warm-start.
- Permissions: notification permission prompt (if reminders are added), microphone/storage prompts avoided where not needed.
- Store compliance: App Tracking Transparency (iOS) if any analytics are added later, Play